Video Intelligence: every model, loss and metric

Study notes to keep beside you in the interview. Concepts, architecture and hand calculations. No code.

How to use these notes

- Section 1 is the map: what is in the project, what is new, what each model is for.
- Sections 2 to 7 take one model or topic each: **what it solves, architecture, loss, metrics, worked numbers**.
- Section 8 lists every setting we chose and why. Section 9 is a question bank with short answers.
- Numbers marked *(from papers / docs)* are published values; check the source if you quote them exactly.

1. The map

Stage	Model / method	Role	Trained loss	Metrics we use or can use
Keyframes	CLIP ViT-B/32	Embed frames; keep a frame only if the scene changed	Symmetric InfoNCE (contrastive)	Compression ratio; keyframe recall (needs labels)
Captions	BLIP base (ViT-B/16 + text decoder)	Describe each keyframe	LM loss (fine-tune); ITC + ITM + LM (pre-train)	CLIPScore, redundancy, unique %; with references: BLEU, METEOR, ROUGE-L, CIDEr, SPICE
Narrative	LLM on Groq (gpt-oss-20b)	Merge captions into one story, EN / AR	Next-token cross-entropy (not trained by us)	Not evaluated (text rewriting only)
Detection	YOLOv8 (n/s/m/l)	Find objects in each frame	CIoU + DFL (box), BCE (class)	mAP@50, mAP@50-95, precision, recall
Tracking	ByteTrack or BoT-SORT (Kalman + Hungarian)	Keep one ID per object over time	None (not trained)	MOTA, IDF1, HOTA, ID switches; proxies without labels
Segmentation	YOLOv8-seg	Pixel mask per object	Box + class + mask BCE	Mask mAP, mask IoU, Dice
Analytics	Counting, dwell time, line crossing, heatmap	Explain what happened	n/a	ID churn, short-track %, mean track length

What is new compared with the original project

Original (kept as is)	New (added)
Frame extraction, CLIP keyframes, BLIP captions, Groq narrative, burned EN / AR captions. TTS and voice-over stay in the folder but are out of scope.	Tracking (YOLOv8 + ByteTrack / BoT-SORT) with analytics dashboard; instance segmentation (YOLOv8-seg); a Full analysis video that merges masks, IDs and captions; evaluation (caption proxies, MOT export, mask IoU / Dice); a Flask web app with four pages and background jobs.

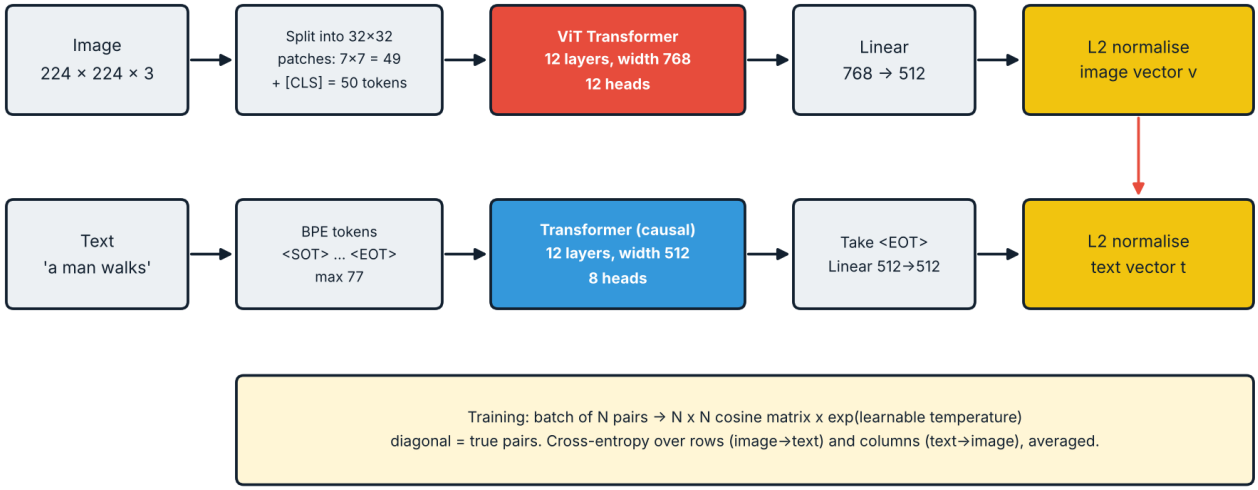
Honest status: all models are pretrained (COCO for YOLO, web image-text pairs for CLIP, BLIP's own data). Nothing is trained or fine-tuned by us. No ground-truth labels exist for our videos, so accuracy numbers (IDF1, MOTA, HOTA, mask mAP) are not yet measured. The web app exports files so they can be.

2. CLIP ViT-B/32 (keyframe selection)

What problem it solves

- A video at 2 fps still has many near-identical frames. Captioning all of them is slow and gives repeated sentences.
- Pixel differences react to lighting, noise and camera shake. We want a change in **meaning**.
- CLIP maps an image to a 512-number vector where similar content gives similar vectors. Cosine similarity of two frames = how alike their content is.

CLIP ViT-B/32: two encoders, one shared 512-d space



Architecture, step by step

- **Image side (ViT-B/32):** 224×224 image cut into 32×32 patches = 7×7 = 49 patches. Each patch is flattened and linearly projected to 768 numbers. A learnable [CLS] token is added, so 50 tokens. Position embeddings are added. 12 Transformer layers (self-attention + MLP), 12 heads. The [CLS] output goes through a linear layer to 512 numbers.
- **Text side:** byte-pair tokens, max 77, start and end tokens. 12-layer Transformer, width 512, 8 heads, causal attention. The vector at the end token goes through a linear layer to 512 numbers.
- Both vectors are L2-normalised (length 1). Then cosine similarity = dot product.
- Why '32' matters: larger patches mean fewer tokens, so B/32 is the fastest CLIP ViT. B/16 is more accurate but about 4× the tokens.
- Trained on about 400 million image-text pairs from the web *(from papers / docs)*.

Loss: symmetric InfoNCE (contrastive)

- Take a batch of N matched (image, text) pairs. Compute the N×N matrix of cosine similarities, multiply by exp(τ) where τ is a learnable temperature.
- Row i = image i against all N texts. The correct answer is column i. Cross-entropy on that row pushes the true pair up and the other N-1 down.
- Do the same for columns (text to image). Average both directions.
- Intuition: it is an N-way multiple-choice question 'which text belongs to this image?' asked in both directions.

```
L_img->txt = -(1/N) Σ_i log( exp(s_ii) / Σ_j exp(s_ij) )
L_txt->img = -(1/N) Σ_i log( exp(s_ii) / Σ_j exp(s_ji) )
L = ( L_img->txt + L_txt->img ) / 2 s_ij = exp(τ) · cos(v_i, t_j)
```

Worked example: InfoNCE with N = 2

Scaled similarity matrix (rows = images, columns = texts): [[4, 1], [0, 3]]. Diagonal = true pairs.

Image 1: softmax of [4, 1] gives $e^4/(e^4+e^1)$ = 54.60/57.32 = 0.953, loss = -ln 0.953 = 0.049. Image 2: row [0, 3] gives 20.09/21.09 = 0.953, loss 0.049.

Text 1: column [4, 0] gives 54.60/55.60 = 0.982, loss 0.018. Text 2: column [1, 3] gives 20.09/22.80 = 0.881, loss 0.127.

L_img->txt = 0.049, L_txt->img = (0.018 + 0.127)/2 = 0.073, so L = (0.049 + 0.073)/2 = **0.061**. Text 2 hurts most because its column has a wrong text scoring 1 vs the true 3.

How we use CLIP in the project

- Embed each sampled frame (2 per second). Keep frame 0. For every next frame compute cos(frame, **last kept** frame). Keep it if cosine < 0.96, then it becomes the new reference.
- Higher threshold = more keyframes (more detail, slower). Lower = fewer keyframes.
- Comparing with the last **kept** frame (not the previous frame) stops slow drift: many tiny changes can add up to a big one.

Worked example: keyframe rule with threshold 0.96

Frames f0..f5. f0 kept. Cosine to f0: f1 = 0.99 skip, f2 = 0.97 skip, f3 = 0.93 **keep** (reference is now f3). Cosine to f3: f4 = 0.98 skip, f5 = 0.94 **keep**.

Keyframes = f0, f3, f5. Compression ratio = 6 frames / 3 keyframes = **2.0**.

Metrics for this stage

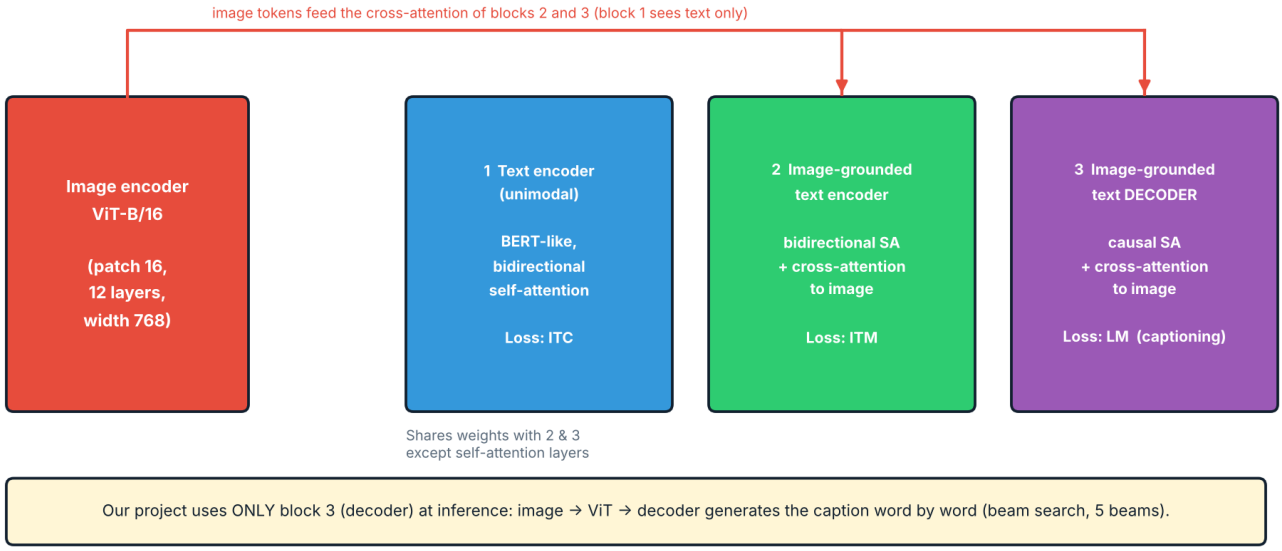
- **Compression ratio** = frames sampled / keyframes kept. Tells how much redundancy was removed. Not a quality score by itself.
- **Keyframe recall / precision** (needs labels): hand-mark the true scene changes. Recall = marked changes that have a keyframe within ±1 s. Precision = keyframes that match a marked change.
- Weak points: 0.96 is a chosen value, not learned. CLIP may ignore small but important changes (a person entering far away).

3. BLIP base (caption generation)

What problem it solves

- Turns a keyframe into one sentence. We use the public checkpoint *Salesforce/blip-image-captioning-base*: a BLIP model with a ViT-B image encoder, fine-tuned for captioning.
- BLIP's idea: web image-text pairs are noisy. It cleans them by letting the model write its own captions and filter bad ones (CapFilt), then trains on the cleaner data.

BLIP: Multimodal mixture of Encoder-Decoder (MED)



Architecture: the MED (Multimodal mixture of Encoder-Decoder)

- **Image encoder:** a ViT with 16×16 patches, 12 layers, width 768. Output = one vector per patch.
- **1. Unimodal text encoder:** BERT-style, bidirectional self-attention, no image. Used for the contrastive loss (ITC).
- **2. Image-grounded text encoder:** same, plus a **cross-attention** layer in every block that lets text tokens look at image tokens. Used for match / no-match (ITM).
- **3. Image-grounded text decoder:** bidirectional self-attention replaced by **causal** self-attention (a word sees only earlier words) plus cross-attention. Used for generation (LM). **This is the part our project runs.**
- Weight sharing: text encoder and decoder share all weights except the self-attention layers, which saves parameters.
- Cross-attention in one line: queries come from the text, keys and values come from the image patches, so each word decides which image regions to read.

Losses in pre-training (three, one per block)

- **ITC (image-text contrastive):** same idea as CLIP's InfoNCE, on the unimodal encoders. BLIP uses a momentum encoder and soft targets (momentum distillation) to cope with noisy pairs.
- **ITM (image-text matching):** binary classifier on the image-grounded encoder: does this text match this image? Trained with **hard negatives**: texts that look similar by ITC but are wrong.
- **LM (language modelling):** the decoder predicts the next word given the image and previous words. Cross-entropy, with label smoothing 0.1 *(from the paper)*.
- **Fine-tuning for captioning (the checkpoint we use):** LM loss only, on COCO captions, with a prompt such as 'a picture of'.

```
L_LM = - Σ_t log p( w_t | w_L_ITM = - [ y log p(match) + (1 - y) log(1 - p(match)) ]
```

Worked example: LM loss for 'a man walks'

The decoder gives these probabilities for the true next words: a = 0.40, man = 0.50, walks = 0.25, end = 0.60.

Loss = -(ln 0.40 + ln 0.50 + ln 0.25 + ln 0.60) = 0.916 + 0.693 + 1.386 + 0.511 = 3.51, per word 0.88. Training lowers this over many images.

CapFilt: bootstrapping clean data

- Start with web pairs (noisy) and human COCO pairs (clean). Fine-tune a **captioner** (decoder) and a **filter** (ITM encoder) on COCO.
- The captioner writes a synthetic caption for each web image. The filter removes bad web captions and bad synthetic ones.
- Pre-train a fresh model on clean web + synthetic + human pairs. Result: better downstream captioning and retrieval.

Inference in our project (settings from the code)

- Image resized to the model's input size, ViT encodes it once.
- Decoder generates left to right. **Beam search with 5 beams**: keep the 5 best partial sentences at every step, choose the best finished one.
- **max_new_tokens = 30** limits length. **repetition_penalty = 1.2** lowers the score of words already used, to avoid 'a man a man a man'.
- Beam search is deterministic: same frame, same caption. Nucleus sampling would be more varied but less stable.

Metrics: with and without human captions

- **Without references (what we run):** CLIPScore, redundancy, unique-caption %.
- **With references (if you had human captions):** BLEU, METEOR, ROUGE-L, CIDEr, SPICE. Details below.

```
CLIPScore = 2.5 · max( cos(image_vec, caption_vec), 0 )
Redundancy = mean over neighbouring caption pairs of |words(a) ∩ words(b)| / |words(a) ∪ words(b)|
Unique % = 100 · (number of distinct captions) / (number of captions)
```

Worked example: CLIPScore

Cosine between a keyframe and its caption = 0.31, so CLIPScore = 2.5 × 0.31 = **0.775**. Average over all keyframes for the video score.

Reading it: higher = the caption matches the image better in CLIP's view. It does not check grammar, Arabic quality or facts that CLIP cannot see.

Reference-based caption metrics (know how each is computed)

Metric	Idea	Formula in words	Weakness
BLEU-n	n-gram precision	Geometric mean of clipped n-gram precisions × brevity penalty BP = exp(1 - r/c) if candidate length c < reference length r	Ignores meaning and recall; synonyms score 0
METEOR	Unigram match with stems and synonyms	F-mean (recall weighted more) × (1 - fragmentation penalty)	Needs language resources; weak for Arabic
ROUGE-L	Longest common subsequence	F-score of LCS/len(candidate) and LCS/len(reference)	Order only, no meaning
CIDEr	TF-IDF weighted n-gram cosine	Average over n=1..4 of cosine between TF-IDF vectors of candidate and references, ×10	Needs several references; main COCO caption metric
SPICE	Scene-graph match	F-score over matched objects, attributes, relations parsed from both captions	Depends on the parser; slow

Worked example: BLEU-2 and ROUGE-L

Candidate: 'a man walks on the street' (6 words). Reference: 'a man is walking on the street' (7 words).

Unigram matches: a, man, on, the, street = 5, so p1 = 5/6. Bigrams: 'a man', 'on the', 'the street' match = 3 of 5, so p2 = 3/5.

Brevity penalty: candidate shorter than reference, BP = exp(1 - 7/6) = 0.847. **BLEU-2 = 0.847 × sqrt(5/6 × 3/5) = 0.847 × 0.707 = 0.599.**

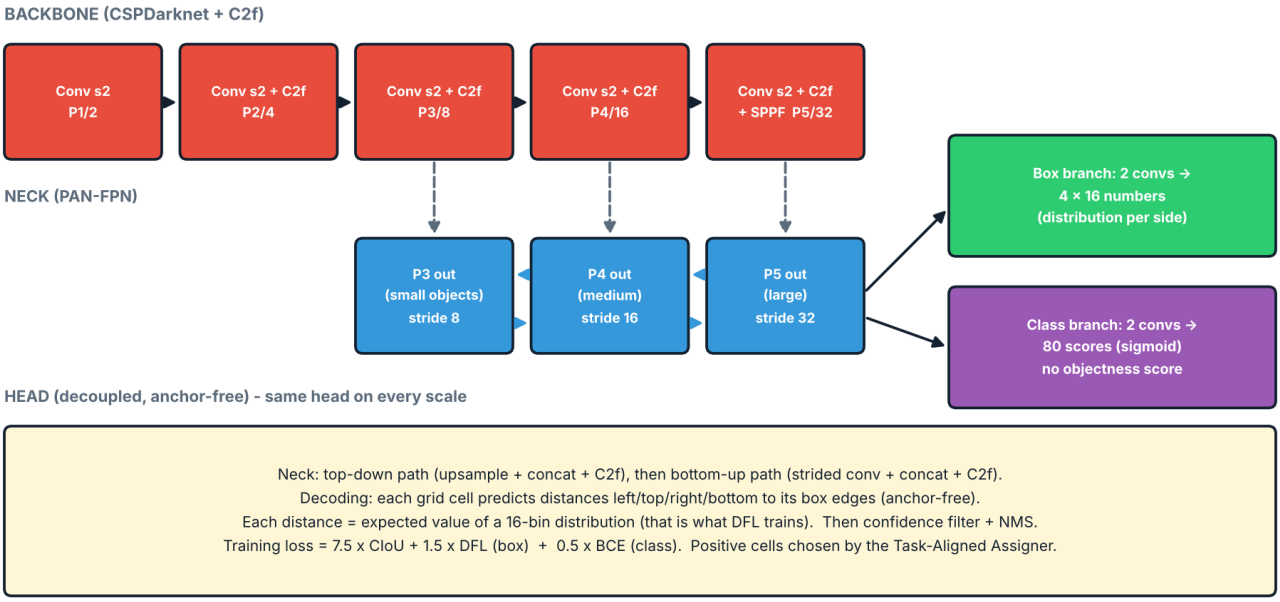
ROUGE-L: longest common subsequence = 'a man on the street' = 5. P = 5/6, R = 5/7, F1 = 2PR/(P+R) = **0.769**.

4. YOLOv8 (detection)

What problem it solves

- Finds every object in a frame: a box, a class and a confidence score. Tracking needs these boxes as input.
- Single-stage: one forward pass gives all boxes. Faster than two-stage detectors (Faster R-CNN) that first propose regions.
- Sizes n, s, m, l (and x). We default to **s**: a balanced choice. Larger = more accurate and slower.

YOLOv8 detection architecture (input 640 × 640)



Architecture, step by step

- **Backbone (CSPDarknet with C2f):** a stack of strided 3×3 convolutions that halve the resolution (strides 2, 4, 8, 16, 32) while adding channels. **C2f block** = split the features in two, pass one part through several small bottleneck blocks, **concatenate every intermediate output**, then a 1×1 conv. More gradient paths and feature reuse than a plain chain, at low cost.
- **SPPF** (spatial pyramid pooling, fast): three max-pools of size 5 applied one after another, all outputs concatenated. Gives a large receptive field cheaply.
- **Neck (PAN-FPN):** features from strides 8, 16, 32 are fused. Top-down path (upsample deep features, concat with shallower ones) gives small layers semantic strength. Bottom-up path (strided conv, concat) passes precise location information back up. Output: three feature maps (P3, P4, P5).
- **Head (decoupled, anchor-free):** separate small conv branches for box and class (classification and localisation want different features). No anchors: each grid cell directly predicts the distance to the four box edges. No objectness score in v8: confidence = the class score.
- **Box as a distribution:** each edge distance is predicted as a probability distribution over 16 bins; the value used = expected value. This is what DFL trains.
- Why three scales: P3 (stride 8) sees small objects, P5 (stride 32) sees large ones.
- **Post-processing:** drop boxes under the confidence threshold (we use 0.3), then NMS (non-maximum suppression) removes duplicate boxes of the same object.

Training loss (three terms)

- **Box loss = CIoU loss.** IoU alone gives no gradient when boxes do not overlap. CIoU adds a centre-distance term and an aspect-ratio term.
- **DFL (distribution focal loss)**: trains the 16-bin distribution so that the probability mass sits on the two bins around the true distance.
- **Class loss = BCE** (binary cross-entropy) per class, each class independent (sigmoid, not softmax).
- **Task-Aligned Assigner (TAL)** decides which grid cells are 'positives' for each ground-truth box, using a score that combines predicted class score and IoU: $t = s^{\alpha} \cdot u^{\beta}$ (defaults $\alpha = 0.5$, $\beta = 6$, top-10

candidates, check your installed config). It aligns classification confidence with localisation quality.

- Total = 7.5 × box + 0.5 × class + 1.5 × DFL (default gains).

```
IoU = |A ∩ B| / |A ∪ B|
L_CIoU = 1 - IoU + ρ²(b, b_gt)/c² + α·v
v = (4/π²) · ( arctan(w_gt/h_gt) - arctan(w/h) )² α = v / (1 - IoU + v)
L_DFL = - [ (y_{i+1} - y)·log S_i + (y - y_i)·log S_{i+1} ] y_i ≤ y ≤ y_{i+1}
L_BCE = - [ y·log p + (1 - y)·log(1 - p) ]
```

Worked example: CIoU

Predicted box A = (0,0)-(10,10). True box B = (5,5)-(15,15). Intersection 5×5 = 25. Union = 100 + 100 - 25 = 175. IoU = 25/175 = 0.143.

Centre distance squared $\rho^2 = 5^2 + 5^2 = 50$. Smallest box enclosing both is 15×15, so $c^2 = 15^2 + 15^2 = 450$. $\rho^2/c^2 = 0.111$.

Same aspect ratio, so v = 0. **L_CIoU = 1 - 0.143 + 0.111 = 0.968.** If the boxes did not overlap, IoU = 0 but the distance term would still give a gradient.

Worked example: DFL

True distance y = 3.4 lies between bins 3 and 4. Weights: bin 3 gets (4 - 3.4) = 0.6, bin 4 gets 0.4. Model's softmax: $S_3 = 0.5$, $S_4 = 0.3$.

L_DFL = -(0.6·ln 0.5 + 0.4·ln 0.3) = 0.6×0.693 + 0.4×1.204 = **0.898**. Lowest when all mass sits on bins 3 and 4 in the 60 / 40 ratio.

Metrics: precision, recall, AP, mAP

- A detection is a **TP** if its class is right and IoU with an unmatched true box ≥ the IoU threshold; otherwise **FP**. Unmatched true boxes are **FN**.
- **Precision** = TP/(TP+FP): of what I predicted, how much is right. **Recall** = TP/(TP+FN): of what exists, how much I found.
- **AP** for one class: sort detections by confidence, walk down the list, record precision and recall, take the area under the (interpolated) precision-recall curve.
- **mAP@50** = mean AP over classes at IoU 0.5. **mAP@50-95** = also average over IoU thresholds 0.50, 0.55, ..., 0.95 (COCO standard). It rewards tight boxes.
- Published COCO val numbers for YOLOv8 detection, mAP@50-95: n 37.3, s 44.9, m 50.2, l 52.9; parameters about 3.2M, 11.2M, 25.9M, 43.7M *(from Ultralytics docs; verify)*.

Worked example: AP from a ranked list

4 true objects. Five detections by confidence: TP, TP, FP, TP, FP.

After each: (precision, recall) = (1/1, 1/4), (2/2, 2/4), (2/3, 2/4), (3/4, 3/4), (3/5, 3/4).

Interpolated precision = highest precision at the same or higher recall: recall 0.25 → 1.0, 0.50 → 1.0, 0.75 → 0.75.

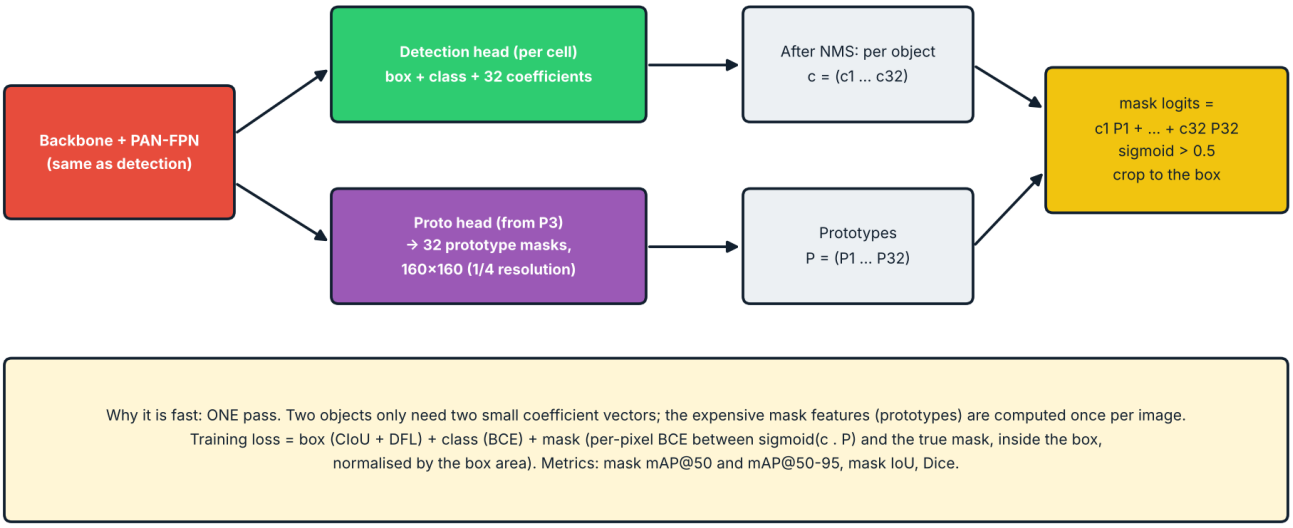
AP = 0.25×1.0 + 0.25×1.0 + 0.25×0.75 + 0 (recall never reaches 1.0) = **0.6875**. COCO samples 101 recall points; the idea is the same.

5. YOLOv8-seg (instance segmentation)

What problem it solves

- A box includes background. A **mask** marks exactly the pixels of each object, and keeps two touching people as two objects (**instance** segmentation).
- Semantic segmentation (U-Net, DeepLab) gives one label per pixel but cannot separate two people. Instance masks can get their own tracking ID.
- Alternatives: Mask R-CNN (accurate, two-stage, slower), SAM / SAM 2 (segments anything, heavy, no class names by itself).

YOLOv8-seg: detection head + prototype masks



Why it is fast: ONE pass. Two objects only need two small coefficient vectors; the expensive mask features (prototypes) are computed once per image.
Training loss = box (CIoU + DFL) + class (BCE) + mask (per-pixel BCE between sigmoid(c . P) and the true mask, inside the box, normalised by the box area). Metrics: mask mAP@50 and mAP@50-95, mask IoU, Dice.

Architecture: what is added to the detector

- Same backbone and PAN-FPN as detection.
- **Proto head:** takes the highest-resolution neck feature (P3) and upsamples to 1/4 of the input (160×160 for 640 input). Output: **32 prototype masks** for the whole image. Think of them as 32 reusable 'building-block' masks, for example 'left side', 'edges', 'top region'.
- **Detection head** now also outputs **32 mask coefficients** per object (besides box and class).
- **Mask of one object** = sigmoid(c1·P1 + c2·P2 + ... + c32·P32), keep pixels above 0.5, **crop to the predicted box**. It is a weighted mix of the building blocks.
- Cost: one matrix multiplication per object. The heavy part (prototypes) is computed once per image, so cost barely grows with the number of objects.
- Idea comes from YOLACT. Limit: mask resolution is 1/4 of the input, so thin details and exact boundaries are approximate.

Loss

- Box (CIoU + DFL) and class (BCE) exactly as in detection.
- **Mask loss:** binary cross-entropy between the predicted mask (sigmoid of c·P) and the true mask, computed inside the box and normalised by the box area so that big objects do not dominate.
- Trained on COCO instance masks (80 classes).

Worked example: one mask pixel

At a pixel, two prototype values P1 = 0.9, P2 = 0.2, other coefficients 0. Coefficients c1 = 4, c2 = -4. Logit = 4×0.9 + (-4)×0.2 = 2.8. sigmoid(2.8) = 0.94 → the pixel belongs to the object.

True mask says 1: BCE = -ln 0.94 = 0.062 (small). If the true label were 0: BCE = -ln(0.06) = 2.8 (large).

Metrics

- **Mask IoU** = overlap of predicted and true mask / their union. **Dice** = 2 × overlap / (sum of both areas). Dice = 2·IoU/(1 + IoU), so Dice is always ≥ IoU.
- **Mask mAP@50 and @50-95:** same procedure as box mAP, but 'IoU' is the **mask** IoU.
- Published COCO val mask mAP@50-95: n 30.5, s 36.8, m 40.8, l 42.6 *(from Ultralytics docs; verify)*.
- Not used here: pixel accuracy and mIoU (for semantic segmentation), boundary F-score (edge quality).

Worked example: IoU and Dice

Predicted mask 50 pixels, true mask 70 pixels, overlap 40. Union = 50 + 70 - 40 = 80. **IoU = 40/80 = 0.50.**

Dice = 2×40/(50+70) = 80/120 = 0.667. Check: 2×0.5/(1+0.5) = 0.667.

What our dashboard shows (descriptive, not accuracy)

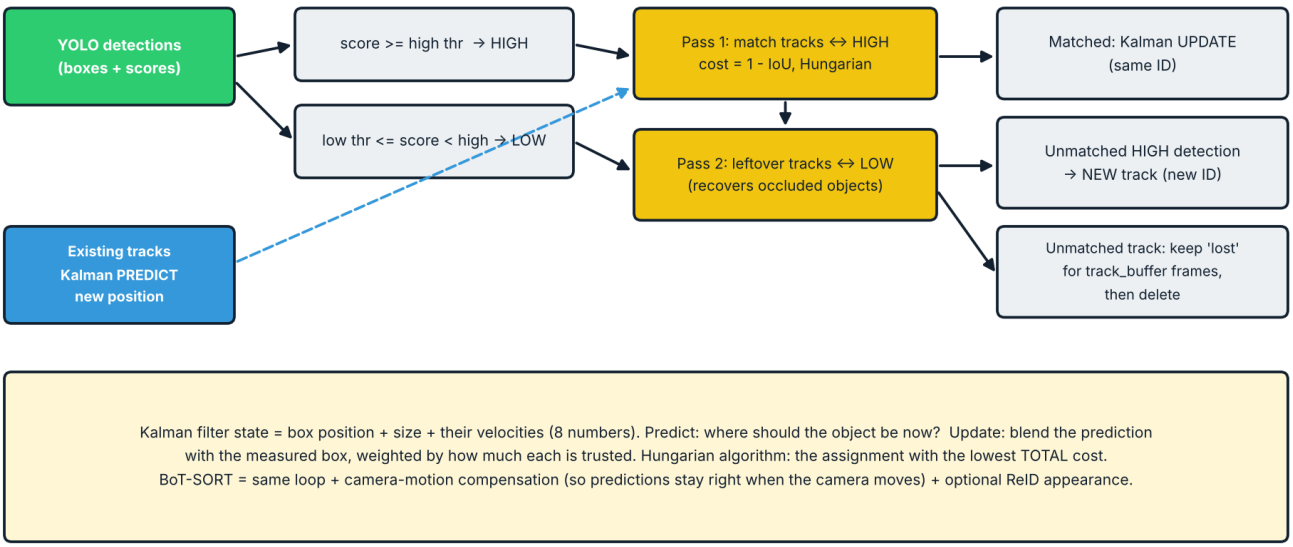
- Instances per class, average confidence, mean and max **area share** = mask pixels of that class / frame pixels × 100, per frame, plotted over time.
- These describe the output. Without ground-truth masks we cannot report mask mAP or IoU.

6. Tracking: ByteTrack and BoT-SORT

What problem it solves

- Detection treats every frame alone. Counting unique objects or measuring time on screen needs the **same object to keep the same ID** across frames.
- Tracking-by-detection: detect every frame, then **associate** new detections with existing tracks.
- Trackers are not neural networks that we train. They are algorithms (Kalman filter + matching). Only the detector has a loss.

ByteTrack: what happens on every frame



Kalman filter state = box position + size + their velocities (8 numbers). Predict: where should the object be now? Update: blend the prediction with the measured box, weighted by how much each is trusted. Hungarian algorithm: the assignment with the lowest TOTAL cost. BoT-SORT = same loop + camera-motion compensation (so predictions stay right when the camera moves) + optional ReID appearance.

Building block 1: the Kalman filter (motion prediction)

- State = box centre, box size and the velocity of each (8 numbers). Assumes roughly constant velocity between frames.
- **Predict:** push the state forward one frame; uncertainty P grows (we know less after moving on).
- **Update:** when a measurement (matched box) arrives, correct the prediction. The **Kalman gain K** decides how much to trust the measurement vs the prediction: high measurement noise → small K, trust the prediction; high prediction uncertainty → large K, trust the measurement.
- Why it helps: the prediction tells where to look for the object next frame, and bridges frames where the detector misses it.

$$\text{Predict: } x' = F\,x \qquad P' = F\,P\,F^{\top} + Q$$

$$\text{Update: } K = P'\,H^{\top}\,(H\,P'\,H^{\top} + R)^{-1} \qquad x = x' + K\,(z - H\,x') \qquad P = (I - K\,H)\,P'$$

Worked example: 1-D Kalman update

Predicted position x' = 10, uncertainty P' = 4. Measurement z = 12 with noise R = 1 (H = 1).

K = 4/(4 + 1) = 0.8. New position = 10 + 0.8×(12 - 10) = **11.6**. New uncertainty = (1 - 0.8)×4 = **0.8**. The detector is trusted more than the prediction, and the estimate is more certain than either.

Building block 2: matching with IoU and the Hungarian algorithm

- Build a cost matrix: rows = predicted track boxes, columns = detections, cost = 1 - IoU.
- The **Hungarian algorithm** finds the one-to-one assignment with the lowest **total** cost (better than greedy picking, which can steal a good match from another track).
- Matches with cost above a threshold are rejected (the boxes do not overlap enough).

Worked example: assignment

Cost matrix (1 - IoU): T1: [D1 0.2, D2 0.9], T2: [D1 0.8, D2 0.3].

Option A: T1-D1 + T2-D2 = 0.2 + 0.3 = **0.5**. Option B: T1-D2 + T2-D1 = 0.9 + 0.8 = 1.7. The algorithm picks A.

ByteTrack (our default): the two-pass idea

- Older trackers (SORT) throw away low-confidence detections. But a partly hidden person often gets a **low score**, not zero.
- **Pass 1:** match all tracks with high-score detections. **Pass 2:** match the **unmatched tracks** with low-score detections. This recovers occluded objects without creating false tracks.
- Unmatched high-score detections start **new tracks**. Unmatched tracks are kept 'lost' for **track_buffer** frames (default about 30; check bytetrack.yaml in your Ultralytics install), then deleted.
- No appearance (ReID) network, so it is very fast. Weakness: two similar objects that cross or a long occlusion can swap IDs.
- Ultralytics call: model.track with persist=True (keep tracker state between frames) and the tracker config file.

BoT-SORT (option) and the older trackers

Tracker	Motion	Appearance	Distinctive idea	Trade-off
SORT	Kalman	No	Kalman + Hungarian on IoU	Fast; many ID switches
DeepSORT	Kalman	Yes: CNN embedding (128-d)	Cosine appearance distance, matching cascade, gating	Fewer switches; slower
ByteTrack	Kalman	No	Uses low-score boxes in a second pass	Fast and strong; can swap IDs after long occlusion
BoT-SORT	Kalman	Optional ReID	Camera-motion compensation (aligns frames) + IoU/ReID fusion	Best when the camera moves; slightly slower
OC-SORT	Kalman, observation-centric	No	Re-updates using observations after occlusion, handles non-linear motion	Good for crowds and dance-like motion

Metrics for tracking (need ground truth in MOT format)

- Ground truth = for every frame, the true boxes with identity numbers. The tracker output is compared by matching boxes with IoU ≥ 0.5.
- **MOTA** counts three error types: FN (missed objects), FP (false boxes) and IDSW (ID switches). Dominated by detection quality; can be negative.
- **MOTP** = average localisation quality (overlap) of matched boxes.
- **IDF1**: match whole **trajectories** to true identities over the entire video, then compute an F1 over identity-correct detections. Focuses on 'does an object keep one ID?'
- **HOTA** = sqrt(DetA × AssA), averaged over IoU thresholds 0.05 to 0.95. **DetA** = detection accuracy = TP/(TP+FN+FP). **AssA** = association accuracy: for every matched detection, how well its predicted track and true track agree over time. It balances detecting well and associating well, and avoids MOTA's detection bias.
- Also: **ID switches**, **fragmentations**, **MT / ML** (mostly tracked / mostly lost trajectories ≥ 80% / ≤ 20% covered), FPS.
- Tools: py-motmetrics (MOTA, IDF1) and TrackEval (HOTA). Standard benchmarks: MOT17, MOT20.

```
MOTA = 1 - ( FN + FP + IDSW ) / GT
IDP = IDTP/(IDTP + IDFP) IDR = IDTP/(IDTP + IDFN) IDF1 = 2·IDTP / ( 2·IDTP + IDFP + IDFN )
DetA = TP / (TP + FN + FP) HOTA_α = sqrt( DetA_α · AssA_α ) HOTA = mean over α
```

Worked example: MOTA and IDF1

Ground truth has 100 object-frames. The tracker missed 8 (FN = 8), produced 5 false boxes (FP = 5) and switched an ID twice (IDSW = 2).

MOTA = 1 - (8 + 5 + 2)/100 = 0.85.

The tracker output 100 - 8 + 5 = 97 boxes. Suppose 84 of them carry the correct identity: IDTP = 84, IDFN = 100 - 84 = 16, IDFP = 97 - 84 = 13.

IDF1 = 2×84 / (2×84 + 13 + 16) = 168/197 = 0.853. A tracker with identical detections but constant ID switches would keep MOTA high but score a much lower IDF1.

Worked example: HOTA

TP = 80, FN = 20, FP = 10: DetA = 80/110 = 0.727. Suppose AssA = 0.90. HOTA at this threshold = sqrt(0.727 × 0.90) = **0.809**. The final HOTA averages many thresholds.

Metrics without ground truth (what our dashboard shows)

- **ID churn** = unique IDs / maximum objects visible at once. Close to 1 = healthy. 3.0 means the tracker issued 3× more IDs than needed: it keeps losing and re-creating objects.
- **Short-track %** = tracks lasting under 1 second. High = flicker or false detections.
- **Mean track length** in seconds.
- These are **hints**. A video with many short-lived objects also has high churn, so they cannot replace IDF1 or HOTA.

Analytics we compute from tracks

- **Unique count per class** = number of distinct IDs (class = the most frequent class seen for that ID).
- **Dwell time** = (last frame - first frame + 1) / fps.
- **Line crossing**: a horizontal line at the middle of the frame. An object crosses when its centre's side of the line changes between two detections (up to down or down to up).
- **Heatmap**: add one count at each object centre in every frame, blur with a Gaussian, colour-map, overlay on the first frame.
- **Trajectory**: the sequence of centres of one ID.

Worked example: dwell time and churn

An object first seen at frame 50 and last seen at frame 199 in a 25 fps video: (199 - 50 + 1)/25 = **6.0 s**.

A video where at most 4 objects appear together but 12 IDs were issued: churn = 12/4 = **3.0**.

7. Remaining parts of the project

LLM narrative (Groq, gpt-oss-20b)

- Takes the per-keyframe captions (after a de-duplication step) and writes one connected narrative in English or Arabic.
- A decoder-only Transformer language model (mixture-of-experts: about 21B parameters in total, about 3.6B active per token *(from the model card; verify)*), trained with next-token cross-entropy. We only call it through an API.
- Risk to mention: an LLM can add details that are not in the captions. Arabic output depends on it, so Arabic quality is not evaluated by our metrics.

Full analysis feature

- One pass of YOLOv8-seg with tracking switched on: masks + IDs from the same detections, so they always agree.
- BLIP keyframe captions are drawn as a caption bar on each frame between their timestamps (English).
- One dashboard: tracking analytics + segmentation area statistics + caption list.

Web app design

- Flask serves four pages. An upload starts a **background thread**; the browser polls a job endpoint for progress. A lock allows one heavy job at a time.
- Output video is re-encoded to **H.264** with ffmpeg because browsers cannot play OpenCV's default mp4v.
- Each feature is a folder with numbered steps (like the original captioning project) and a pipeline file that runs them in order.

8. Every setting we chose, and why

Setting	Value	Why / effect
Frame sampling	2 per second	Enough to catch scene changes; far cheaper than all frames
CLIP keyframe threshold	cosine < 0.96	Higher = more keyframes, lower = fewer; chosen empirically, not learned
Reference frame for the rule	last kept frame	Prevents slow drift missing a big total change
BLIP decoding	beam search 5, max 30 new tokens, repetition penalty 1.2	Stable, short, non-repetitive captions
Detector size	YOLOv8 s (n / m / l selectable)	Balanced accuracy vs speed; n for speed, m / l for accuracy
Detection confidence	0.3	Lower finds more objects but more false positives; higher the opposite
Tracker	ByteTrack (BoT-SORT selectable)	Fast and strong; BoT-SORT if the camera moves
Trail length	40 positions	Only for drawing
Line crossing	horizontal line at 50% height	Simple default; the position is a parameter in the analytics code
Mask opacity	0.45	Visualisation only
Output encoding	H.264, yuv420p	Plays in every browser

Known limits (say them yourself first)

- Everything is pretrained: only COCO's 80 classes are detected and segmented; no fine-tuning on our domain.
- No ground truth, so IDF1 / MOTA / HOTA / mask mAP / CIDEr are not measured. The app exports a MOT file; the evaluation helper computes MOTA and IDF1 with py-motmetrics; HOTA needs TrackEval.
- ByteTrack has no appearance model: ID swaps after long occlusions or crossings.
- The keyframe threshold and detection confidence are manual choices.
- Arabic captions come from an LLM working on English captions, so BLIP's errors pass through.
- The numbers a video gives (counts, dwell time) are only as good as the detector.

9. Question bank with short answers

Why CLIP and not frame differencing or histograms?

They react to pixels (light, noise). CLIP compares meaning, so a keyframe appears only when the scene content changes.

What does CLIP's loss do?

Symmetric InfoNCE: in a batch, each image must pick its own text among N, and each text its own image. Matching pairs get high cosine, the rest low.

Why compare with the last kept frame?

Comparing consecutive frames misses slow drift: each step looks similar but the sum is a new scene.

How does BLIP differ from CLIP?

CLIP only embeds and compares (no text generation). BLIP adds a cross-attention decoder that generates text, plus ITC, ITM and LM losses.

What is cross-attention in BLIP?

Queries from the text tokens, keys and values from the image patches. Each word chooses which image regions to read.

What is CapFilt?

A captioner writes synthetic captions for web images; a filter removes noisy ones; the model is retrained on the cleaned set.

Beam search vs sampling?

Beam keeps the 5 best partial sentences, deterministic and fluent. Sampling is random and more varied but less stable.

How do you evaluate captions without references?

CLIPScore (image-text cosine), plus redundancy and unique-caption %. With references you would add CIDEr, BLEU, METEOR, ROUGE-L, SPICE.

Why YOLOv8?

Single-stage, fast, anchor-free, good accuracy, sizes n to l, and the same family gives both detection and segmentation.

What is C2f?

A split-transform-concatenate block: part of the features passes through several bottlenecks, all intermediate outputs are concatenated. Richer gradient flow at low cost.

What is anchor-free?

No predefined box shapes. Each cell predicts distances to the four box edges directly.

What is DFL?

A loss that trains each edge distance as a 16-bin probability distribution; the box edge is its expected value. It handles ambiguous boundaries better than one number.

Why CIoU?

IoU gives zero gradient for non-overlapping boxes. CIoU adds centre distance and aspect ratio so the loss is always informative.

What is mAP@50-95?

Mean over classes of AP, averaged over IoU thresholds 0.50 to 0.95. Rewards tight boxes, stricter than mAP@50.

How does the segmentation mask get built?

32 prototype masks per image, 32 coefficients per object. Mask = sigmoid(coefficients · prototypes), cropped to the box.

Instance vs semantic segmentation?

Semantic labels pixels by class only; instance also separates objects of the same class.

Why not SAM?

Heavier, no class labels on its own. It is a good next step, for example to refine masks from YOLO boxes.

Why track?

To count unique objects and measure time. Detection alone counts one person many times.

What does the Kalman filter do?

Predicts where each object will be next frame and blends that with the measurement according to their uncertainties.

Why Hungarian instead of greedy?

It minimises the total cost over all pairs; greedy can take a good match away from another track.

What makes ByteTrack special?

A second matching pass with low-confidence detections, recovering partly hidden objects.

BoT-SORT vs ByteTrack?

BoT-SORT adds camera-motion compensation and optional ReID. Better with moving cameras, a bit slower.

MOTA vs IDF1 vs HOTA?

MOTA: error counting, detection-heavy. IDF1: identity consistency over time. HOTA: geometric mean of detection and association accuracy, balanced.

How would you evaluate your tracker?

Label 20-30 seconds in MOT format (or use MOT17), run the pipeline, export tracks_mot.txt, compute MOTA / IDF1 with py-motmetrics and HOTA with TrackEval.

What if there are many ID switches?

Raise detection quality (larger model), raise confidence or track buffer, use BoT-SORT with ReID, or DeepSORT-style appearance matching.

What would you do next?

Label a short clip and report IDF1 / HOTA and mask IoU; fine-tune the detector on our domain; try SAM 2 for mask tracking; add reference captions for CIDEr.

Biggest weakness of the project?

Pretrained only and no ground truth, so no accuracy numbers on our own videos yet. Say it first, then show the plan above.

Numbers to fill in after you run the app

Item	Your value
Frames sampled / keyframes / compression ratio	
Mean CLIPScore	
Detector size, hardware, processing fps	
Unique objects / ID churn / short-track %	
IDF1 / MOTA / HOTA on a labelled clip (optional)	
Mask mAP or IoU on a labelled clip (optional)	